

Our experiments show that on versions where CVE-2023-41991 is present, the PerfectCheater bypass remains effective. Before diving into the details, let us first examine how the two bypass techniques differ.

Key Premise: After SHA256 was introduced, for compatibility reasons, when a signature contains both SHA1 and SHA256 hashes, CoreTrust validates the SHA1 hash but executes the binary pointed to by the SHA256 hash.

CVE–2023–41991:

Affected Versions: iOS 14.0Beta2—16.7 RC (20H18) and iOS 17.0

The attacker constructs a malicious signature with the following structure through a series of complex steps:

Type	SHA 1 Signature	SHA 256 Signature
Hash Value	Safari's	TrollStore's
Signer	Apple Inc.	Other
Binary	Safari	TrollStore

When CoreTrust validates, it discovers that SHA1 is a system App's value, so grants high trust, grants all entitlements it declares. After AMFI's adjudication, the binary pointed to by SHA256 is executed, so a malicious app can gain any privileges it declares

Mitigation: The fix was to unify the TeamID: both the SHA1 and SHA256 signatures must now be issued by the same signer. This closed the bypass vector exploited by CVE-2023-41991.

PerfectCheater:

Affected Versions: iOS 14.0Beta2–17.0.1, and additionally includes iOS 16.7 – 16.7.x and iOS 17.0.1.

The attacker constructs a malicious signature with the following structure through a series of complex steps:

Type	SHA 1 Signature	SHA 256 Signature
Hash Value	Safari's	TrollStore's
Signer	Other	Other
Binary	Safari	TrollStore

When CoreTrust validates, it validates TeamID first, discover that TeamID was the same, fulfills the condition. The vulnerability is that CoreTrust verifies whether the SHA1 and SHA256 hashes are signed by the same signer, but does not verify the identity of that signer. Then discover that SHA1 is a system App's value, so grants high trust, grants all entitlements it declares, so a malicious app can gain any privileges it declares.

Although this vulnerability remains unfixed to this day, the introduction of the trust cache in iOS 17.0.2 effectively blocks it. Specifically, CoreTrust now first checks whether the binary hits the trust cache. If it does, the declared entitlements are granted directly. If not, the binary proceeds to CoreTrust validation.

When an attacker follows the same method described above, the trust cache considers the binary as invalid, while CoreTrust considers it valid. During AMFI adjudication, a conservative decision is made for safety reasons — AMFI treats the binary as invalid, and the exploit fails.

As a result, this vulnerability is only effective on versions that have not yet introduced the trust cache. Among those, iOS 16.7.x and 15.8.x are still receiving security updates. However, 15.8.x does not patch CVE-2023-41991, and overlaps with versions already affected by that CVE, offering no additional value. Therefore, the practical value of this vulnerability is primarily in iOS 16.7 (excluding 16.7 RC) through 16.7.x, and iOS 17.0.1.